

PayFlow - Cursor Build Blueprint

1. Product

PayFlow is a SaaS for Indian freelancers and small agencies to create invoices, collect payments through an established payment gateway, track outstanding invoices, send reminders, and view revenue analytics.

Positioning

“Get paid. Without chasing clients.”

Initial customer

Indian freelancers and small agencies: web developers, designers, video editors, SEO/marketing agencies, consultants, photographers, software agencies.

Core pain

Business owners lose time tracking who owes money, sending invoices, and following up on overdue payments.

Business model

Free: 5 invoices/month, basic dashboard, payment links. Pro: INR 999/month, unlimited invoices, payment links, reminders, PDF invoices, receipts, analytics, customer management. Business: INR 2,499/month, Pro plus team members, advanced analytics, recurring invoices, WhatsApp notifications, priority support.

The INR 50,000/month example is 20 Pro customers (INR 19,980) + 12 Business customers (INR 29,988) = INR 49,968. This is a target, not a guarantee.

2. Product principles

1. Payments are handled by an established gateway; PayFlow is not a payment processor.
2. Never trust the browser to declare a payment successful. Payment status is finalized by verified gateway webhooks.
3. Webhook processing must be idempotent.
4. Every tenant-scoped database query must enforce business/tenant ownership.
5. Build the smallest sellable MVP before advanced features.
6. Prefer correctness, security, simplicity, and observability over premature complexity.

3. MVP scope

Build these first: - Marketing landing page - Registration/login/logout - Email verification and password reset - Business profile - Customer CRUD - Invoice CRUD - Invoice line items, tax, discount, due date, statuses - Professional invoice PDF - Public invoice/payment page - Payment gateway integration in test/sandbox mode first - Backend webhook verification - Idempotent payment recording - Automatic invoice status transition to PAID - Payment and invoice notifications by email - Dashboard: revenue, collected, pending, overdue, recent invoices/payments - Revenue chart and payment status chart - Subscription/paywall skeleton for Free/Pro/Business - Basic admin/operations view - Audit logging for financial mutations - Production deployment configuration - Monitoring/error tracking

Do not include in MVP unless needed for validation: - WhatsApp automation - recurring payment automation - mobile app - AI assistant - accounting integrations - public developer API - Elasticsearch - complex multi-region infrastructure

4. Recommended stack

Frontend

- Next.js (App Router) + TypeScript
- Tailwind CSS
- shadcn/ui
- React Hook Form + Zod
- TanStack Query where client-side server state is needed
- Recharts for charts
- Three.js only on the marketing experience, not the operational dashboard

Backend

- Node.js + TypeScript
- Express
- Zod request validation
- Mongoose or MongoDB Node driver
- Auth using secure server-side sessions / HTTP-only secure cookies or a mature auth library
- Background jobs via a queue/worker system when reminders are introduced

Data/infrastructure

- MongoDB Atlas
- Redis only when needed for rate limiting, queues, caching, or sessions
- Object/file storage for invoice PDFs if generated files are persisted
- Email provider for transactional email

- Established payment gateway such as Razorpay or another compliant provider; use its sandbox first

Deployment

- Web: Vercel or equivalent
- API: Render/Railway/AWS or equivalent
- Database: MongoDB Atlas
- Domain: custom domain
- HTTPS enabled
- CI/CD via GitHub Actions or platform-native integration

5. High-level architecture

Internet -> Next.js web app -> Node/Express API -> MongoDB Atlas -> Payment gateway -> verified webhook -> Express API -> email provider -> background queue/worker for reminders later

Suggested repo: payflow/ apps/ web/ api/ packages/ types/ validation/ config/ docs/ tests/ .env.example package.json

For a first iteration, it is acceptable to keep packages in a simpler structure and refactor only when duplication appears.

6. Multi-tenant model

A business is the tenant boundary. A user owns or belongs to one or more businesses in a future version; MVP can support one primary business per user.

Core collections: - users - businesses - customers - invoices - payments - subscriptions - notifications - audit_logs

Potential later collections: - invoice_files - recurring_invoices - team_members - usage_counters - webhook_events

User

`_id email passwordHash emailVerifiedAt createdAt updatedAt`

Business

`_id ownerId name logoUrl email phone address currency invoicePrefix createdAt updatedAt`

Customer

`_id businessId name email phone company createdAt updatedAt`

Invoice

`_id businessId customerId invoiceNumber items[] { description, quantity, unit-Price, lineTotal } subtotal taxAmount discountAmount totalAmount currency status: DRAFT | SENT | PENDING | PAID | OVERDUE | CANCELLED issueDate dueDate publicToken/paymentPageToken paymentLink sentAt paidAt createdAt updatedAt`

Payment

`_id businessId invoiceId gateway gatewayOrderId gatewayPaymentId gatewayEventId amount currency status: CREATED | AUTHORIZED | CAPTURED | FAILED | REFUNDED paidAt rawEventMetadata only where legally/operationally appropriate; minimize sensitive storage createdAt updatedAt`

Subscription

`_id businessId plan: FREE | PRO | BUSINESS provider providerCustomerId providerSubscriptionId status currentPeriodStart currentPeriodEnd createdAt updatedAt`

Notification

`_id businessId type recipient channel status providerMessageId relatedInvoiceId error createdAt`

AuditLog

`_id businessId actorUserId action entityType entityId metadata createdAt`

7. Important indexes

Create and verify indexes based on actual query patterns. Initial examples:
- `users.email` unique - `businesses.ownerId` - `customers.businessId` + `createdAt` - `invoices.businessId` + `createdAt` desc - `invoices.businessId` + `status`
- `invoices.businessId` + `dueDate` - `invoices.businessId` + `customerId` + `createdAt` desc - `payments.businessId` + `createdAt` desc - `payments.invoiceId` - `payments.gatewayPaymentId` unique when provider semantics allow - `webhook event/gateway event ID` unique for idempotency

Do not add indexes blindly. Monitor query performance and index growth.

8. Authentication and authorization

- Hash passwords with a modern password hashing algorithm such as Argon2id or bcrypt with appropriate cost.
- Use secure, HTTP-only, same-site cookies for browser sessions.

- Use CSRF protection where cookie-based auth and state-changing cross-site requests require it.
- Verify email before enabling sensitive account functions.
- Rate limit login, register, password reset, public payment pages, and web-hook endpoints where appropriate.
- Implement centralized authorization middleware/service.
- Never trust businessId, userId, customerId, or invoiceId from the client without verifying ownership.
- Never expose secret keys to the browser.

9. Invoice flow

User -> Create Invoice -> validate input -> calculate subtotal/tax/discount -> persist invoice -> show invoice detail -> Generate PDF -> Send invoice -> Create payment order/link.

Calculations must be deterministic and server-side validated. Never accept a client-provided total as authoritative.

Invoice status rules

DRAFT: editable, not sent. SENT/PENDING: sent to customer; payment not captured. PAID: verified successful capture. OVERDUE: dueDate passed and not PAID/CANCELLED. CANCELLED: manually cancelled under allowed rules.

Avoid impossible transitions. Keep state transition logic in one service/module.

10. Payment flow

1. Business creates invoice.
2. Backend validates invoice and amount.
3. Backend creates an order/payment object with the payment provider.
4. Customer opens public payment page.
5. Customer uses provider checkout.
6. Provider processes payment.
7. Provider sends signed webhook to API.
8. API verifies webhook signature.
9. API identifies event and checks idempotency.
10. API retrieves/validates payment data as supported by provider.
11. API checks invoice, business, amount, currency, and current payment state.
12. API records payment.
13. API updates invoice to PAID in a safe transaction/update pattern.
14. API writes audit log.
15. API enqueues email receipt/notification.
16. Dashboard reads authoritative database state.

Do not mark the invoice PAID solely from a browser callback.

Webhook idempotency

Persist gateway event IDs or another provider-supported unique identifier. Process a repeated event as a no-op after the first successful processing. If concurrent delivery is possible, use a unique database constraint/atomic claim pattern.

11. Public payment page

Route concept: /pay/:securePublicKey

Display only what is necessary: - business name/logo - invoice number - item summary - total - currency - due date/status - pay button

Never expose private internal identifiers or unrelated business data.

12. API contract

Suggested routes: POST /api/auth/register POST /api/auth/login POST /api/auth/logout POST /api/auth/verify-email POST /api/auth/forgot-password POST /api/auth/reset-password

GET /api/business PATCH /api/business

GET /api/customers POST /api/customers GET /api/customers/:id PATCH /api/customers/:id DELETE /api/customers/:id

GET /api/invoices POST /api/invoices GET /api/invoices/:id PATCH /api/invoices/:id DELETE /api/invoices/:id POST /api/invoices/:id/send POST /api/invoices/:id/payment-order POST /api/invoices/:id/pdf

GET /api/payments GET /api/payments/:id

POST /api/webhooks/payment

GET /api/analytics/overview GET /api/analytics/revenue GET /api/analytics/payment-status

GET /api/subscription POST /api/subscription/checkout POST /api/subscription/webhook

Every route must specify authentication, validation, authorization, response shape, and error behavior.

13. API response standard

Use a consistent shape, for example: { data: ..., error: null, meta: ... }

Errors: { data: null, error: { code, message, fieldErrors? }, meta: null }

Do not leak stack traces or internal provider/database errors to customers.

14. UI information architecture

Marketing: / /pricing /features /security /contact /legal/terms /legal/privacy

Auth: /login /register /verify-email /forgot-password /reset-password

App: /app/dashboard /app/invoices /app/invoices/new /app/invoices/:id
/app/customers /app/customers/:id /app/payments /app/analytics /app/settings
/app/billing

Public: /pay/:token

Admin (protected separately): /admin /admin/users /admin/businesses /ad-
min/payments /admin/webhooks /admin/errors

15. Dashboard UI

Desktop layout: - left sidebar - top bar with search/notifications/profile - main
content with KPI cards

Top KPI cards: - Total Revenue - Collected - Pending - Overdue

Primary chart: - Revenue over last 7/30/90 days or current month - line or area
chart

Secondary chart: - Payment status breakdown - donut chart: Paid, Pending,
Overdue

Tables: - recent invoices - recent payments - overdue invoices

Actions: - Create invoice - Add customer - Send reminder

Mobile: - collapsible navigation - KPI cards stacked - simplified charts - tables
become cards or horizontally scrollable when appropriate

16. Chart implementation

Use Recharts. Required charts: 1. Revenue over time - line/area chart. 2.
Payment status - donut chart. 3. Monthly collection comparison - bar chart in
a later release.

Chart data must come from aggregate API endpoints instead of loading every
invoice into the browser. Use fixed server-generated date ranges and timezone-
aware date boundaries.

17. UX rules

- Financial values should always show currency and two-decimal precision
where appropriate.
- Show explicit loading, empty, success, and error states.
- Destructive actions require confirmation.

- Show invoice status consistently using text plus accessible visual treatment.
- Forms should validate both client-side and server-side.
- Avoid over-animation. Three.js belongs on the public marketing experience only.
- Use accessible keyboard navigation, labels, focus states, semantic HTML, and sufficient contrast.

18. Landing page

Hero headline: “Get Paid. Without Chasing.” Subheadline: “Create professional invoices, send payment links, track outstanding payments, and automate reminders.” CTAs: “Start Free” and “See How It Works”

Sections: - product demo/dashboard visual - invoice/payment link flow - features - social proof/testimonial placeholders only until real customer evidence exists - pricing - FAQ - security/trust - final CTA

Three.js may be used for a subtle lightweight payment/invoice visual. It must not delay the page or block the CTA.

19. Email events

- Welcome/verify email
- Invoice sent
- Payment successful receipt
- Business payment notification
- Reminder before due date
- Overdue reminder
- Subscription confirmation

Use a transactional email provider. Store provider IDs/statuses, not unnecessary message content or secrets.

20. Reminder engine (Phase 2)

Preferred architecture: Scheduler -> queue -> worker -> notification provider -> record notification result.

Example schedule: - 1 day before due date - due date - 2 days overdue - configurable later

Prevent duplicate reminders with a deterministic reminder key and unique constraint.

21. Subscription billing

Separate two money flows: A. Customer pays the business’s invoice. B. Business pays PayFlow’s SaaS subscription.

Do not mix these records. Subscription webhooks should update plan/status only after verification.

22. Admin/operations

Protected admin view should expose: - signups - active businesses - plans - invoice counts - payment volume - successful/failed payments - webhook failures - application errors - support flags

Never allow admin UI to bypass auditability. Sensitive actions should create audit logs.

23. Security checklist

- HTTPS only
- Secure cookies
- CSRF protection as applicable
- CORS restricted to known origins
- Rate limiting
- Input validation
- Output encoding as appropriate
- Secure headers
- Password hashing
- Email verification
- Access control on every tenant resource
- Webhook signature verification
- Idempotency
- Secrets via environment/secret manager
- No secrets in source control
- No sensitive data in client logs
- Dependency updates and vulnerability scanning
- Minimal PII storage
- Backups and recovery plan for production database

24. Performance optimization

Frontend: - server-render public pages when useful - lazy load non-critical components - optimize images - avoid shipping Three.js to dashboard pages - cache static assets - paginate invoice/customer tables - debounce search

Backend: - use indexed queries - paginate list endpoints - use lean/read-optimized queries where appropriate - aggregate analytics in MongoDB rather than transferring raw records - avoid N+1 queries - add response compression where appropriate - use caching only after measuring a need

Database: - create indexes for observed access patterns - archive or partition data only when justified by actual scale - monitor slow queries

25. Reliability

- Structured logs with request IDs/correlation IDs
- Central error handling
- Health endpoint: GET /health
- Readiness endpoint if deployment requires it
- Payment webhook retry handling
- Dead-letter/failed job handling when queues are introduced
- Automated database backups
- Alerting on payment/webhook errors

26. Testing plan

Unit: - invoice calculation - tax/discount logic - status transitions - authorization helpers - webhook signature validation helper

Integration: - register/login - customer CRUD - invoice creation - payment order creation - webhook -> payment -> invoice PAID - duplicate webhook remains idempotent

E2E with Playwright: Register -> verify -> login -> customer -> invoice -> public payment page -> sandbox payment -> dashboard updated.

Security tests: - cross-business invoice access must fail - unauthorized admin access must fail - malformed webhook must fail - replayed webhook must not double-book payment

27. Observability

Capture: - API latency - 4xx/5xx rate - payment success/failure rate - webhook success/failure/retry rate - signup conversion - invoice creation conversion - first-payment conversion - paid subscription conversion

The minimum useful funnel is: visitor -> signup -> business created -> invoice created -> payment link opened -> payment succeeded -> paid subscription.

28. SEO and marketing performance

Public marketing pages: - metadata - Open Graph - structured data only where accurate - sitemap - robots.txt - semantic headings - optimized images - Core Web Vitals awareness

Do not put private app pages into search indexes.

29. Environment variables

Use an .env.example containing placeholders only, such as: NODE_ENV= PORT= WEB_ORIGIN= API_ORIGIN= MONGODB_URI= SESSION_SECRET= EMAIL_PROVIDER_API_KEY= PAYMENT_PROVIDER_KEY_ID=

```
PAYMENT_PROVIDER_KEY_SECRET= PAYMENT_WEBHOOK_SECRET=
SUBSCRIPTION_PROVIDER_KEY_ID= SUBSCRIPTION_PROVIDER_KEY_SECRET=
SENTRY_DSN= STORAGE_BUCKET= STORAGE_ACCESS_KEY=
STORAGE_SECRET_KEY=
```

Names can be adapted to the selected providers.

30. CI/CD

On pull request: - install - typecheck - lint - unit tests - build

On main branch: - deploy web - deploy API - run health checks

Use separate development/sandbox and production payment credentials.

31. Cursor workflow

Do not ask Cursor to generate the entire application in one shot. Use vertical slices and test after each.

Recommended sequence: 1. Initialize repo and architecture. 2. Configure TypeScript, linting, formatting, env handling. 3. Create database connection and health endpoint. 4. Implement auth. 5. Implement Business model and authorization. 6. Implement Customer CRUD. 7. Implement Invoice model/calculation service. 8. Implement invoice UI and PDF. 9. Implement payment-provider abstraction in backend. 10. Integrate provider sandbox check-out. 11. Implement signed webhook endpoint. 12. Add webhook idempotency. 13. Build payment/public invoice page. 14. Build dashboard aggregates and charts. 15. Add email notifications. 16. Add pricing/subscription skeleton. 17. Add admin/operations. 18. Add tests and harden security. 19. Deploy staging. 20. Run sandbox end-to-end flow. 21. Deploy production only after verification.

Cursor rules

- Before changing code, inspect relevant files and summarize the intended changes.
- Prefer small commits/patches.
- Do not introduce a new library unless there is a clear reason.
- Reuse existing components and validation.
- Never hardcode secrets.
- Never disable TypeScript/lint errors to make builds pass.
- Do not modify payment security rules without explicit review.
- After each feature, run the smallest relevant tests.

32. First implementation prompt for Cursor

“You are implementing PayFlow, a multi-tenant SaaS for Indian freelancers and small agencies. Read the repository before changing anything. Set up a

production-oriented monorepo with Next.js + TypeScript for the web app and Node.js + Express + TypeScript for the API. Configure shared types/validation only where helpful. Add environment validation, linting, formatting, and a health endpoint. Add MongoDB connection support but do not create business features yet. Do not add payment integration yet. Keep the architecture simple, typed, testable, and ready for incremental development. Output the files changed and how to run the project locally.”

33. Second prompt: auth

“Implement authentication for PayFlow using secure HTTP-only cookies/sessions or a mature auth library. Add register, login, logout, email verification, and password reset. Hash passwords securely. Add request validation, rate limiting hooks, centralized error handling, and authorization middleware. Do not put secrets in source code. Add unit/integration tests for login and unauthorized access. Do not implement payments yet.”

34. Payment integration prompt template

“Implement the payment provider behind a provider abstraction. Use sandbox credentials only. The server creates payment orders from authoritative invoice totals. Never trust client-submitted amount or success status. Implement a signed webhook endpoint, strict validation, event idempotency, duplicate-event protection, amount/currency checks, and safe invoice/payment state transitions. Add integration tests using mocked provider events. Document exactly which environment variables and provider dashboard settings are required.”

35. Acceptance criteria for MVP

The MVP is ready for pilot customers when:

- A user can register and log in securely.
- A business can create and manage customers.
- A business can create a mathematically correct invoice.
- A customer can open a public invoice/payment page.
- A sandbox payment can be completed through the provider.
- A verified webhook changes the invoice to PAID.
- Replaying the webhook does not duplicate the payment.
- A payment receipt/notification is created.
- Dashboard KPIs match database truth.
- A user cannot access another business’s records.
- Tests cover critical financial logic.
- Production configuration does not expose secrets.
- Monitoring can detect API/payment/webhook failures.

36. Product roadmap after MVP

Phase 2: - automated reminders - recurring invoices - team members - custom branding - WhatsApp notifications - advanced analytics

Phase 3: - accounting integrations - expense tracking - public API - multi-currency/internationalization where legally/provider supported - mobile app - partner/referral program

Phase 4: - vertical-specific workflows for gyms, tutors, consultants, agencies, etc.
- deeper automation and integrations based on real customer demand.

37. Go-to-market

Initial market: India. Do not depend on organic traffic for the first customers.

Pilot offer: “Early access for the first 20 businesses: Pro at INR 999/month.”

Sales questions: - How do you currently invoice clients? - How do you track unpaid invoices? - How do you remind customers? - How long do clients typically take to pay? - What would make you pay for a simpler system?

The goal is to get the first few paying businesses and use their feedback to prioritize features.

38. Engineering philosophy

Ship the smallest trustworthy version. Financial software should favor correctness over cleverness. Treat payment state as authoritative data, not UI state. Make important operations auditable. Measure before optimizing. Build for one customer segment first, then broaden.